

Git & GitHub Complete Beginner to Intermediate Guide

A practical guide to learning Git and GitHub with the most commonly used commands and workflows.

What is Git?

Git is a distributed version control system that helps developers:

- Track changes in source code
 - Collaborate with other developers
 - Maintain different versions of a project
 - Restore previous versions if something goes wrong
 - Manage multiple features simultaneously using branches
-

What is GitHub?

GitHub is a cloud-based platform that hosts Git repositories.

It allows you to:

- Store repositories online
 - Collaborate with teams
 - Review code using Pull Requests
 - Track issues and bugs
 - Automate deployment using GitHub Actions
-

Installing Git

Download Git from:

<https://git-scm.com>

After installation, verify it:

```
git --version
```

Example output:

```
git version 2.50.0
```

Configure Git (First Time Only)

Configure your username:

```
git config --global user.name "Mesbah"
```

Configure your email:

```
git config --global user.email "monline1122@gmail.com"
```

Check configuration:

```
git config --list
```

Create a New Repository

Navigate to your project folder.

Initialize Git:

```
git init
```

This creates a hidden `.git` directory.

Clone an Existing Repository

HTTPS

```
git clone https://github.com/username/project.git
```

SSH

```
git clone git@github.com:username/project.git
```

Git Workflow

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Local Repository
  ↓
git push
  ↓
GitHub Repository
```

Check Repository Status

```
git status
```

Shows:

- Modified files
 - New files
 - Deleted files
 - Staged files
 - Branch information
-

Add Files

Add one file:

```
git add index.html
```

Add multiple files:

```
git add file1 file2
```

Add everything:

```
git add .
```

Commit Changes

```
git commit -m "Initial website ready"
```

A commit is a snapshot of your project.

View Commit History

View all commits:

```
git log
```

Compact view:

```
git log --oneline
```

Last 5 commits:

```
git log --oneline -5
```

Remove Files from Staging Area

```
git restore --staged index.html
```

This removes the file from staging without deleting your changes.

Ignore Files

Create:

```
.gitignore
```

Example:

```
.env  
node_modules/  
vendor/  
dist/  
*.log
```

Git will ignore these files.

Keep Empty Folders

Git doesn't track empty folders.

Create:

```
.gitkeep
```

inside the folder to preserve it.

Branches

Create branch:

```
git branch feature-login
```

View branches:

```
git branch
```

Switch branch:

```
git switch feature-login
```

Create and switch:

```
git switch -c feature-login
```

Merge Branches

Switch to main branch:

```
git switch master
```

or

```
git switch main
```

Merge feature branch:

```
git merge feature-login
```

If Git opens Vim:

```
Press ESC
:wq
Enter
```

Delete Branch

```
git branch -d feature-login
```

Force delete:

```
git branch -D feature-login
```

Resolve Merge Conflicts

When conflicts occur:

1. Open the conflicting file.
2. Edit manually.
3. Remove conflict markers.
4. Save file.

Then:

```
git add .
git commit
```

Git Stash

Temporarily save changes:

```
git stash
```

View stashes:

```
git stash list
```

Restore latest stash:

```
git stash pop
```

Restore but keep stash:

```
git stash apply
```

Delete stash:

```
git stash drop
```

Delete all stashes:

```
git stash clear
```

Git Tags

Tags mark important releases.

View tags:

```
git tag
```

Annotated tag:

```
git tag -a v1.0 -m "My Release 1"
```

Lightweight tag:

```
git tag v1.1
```

Push tags:

```
git push origin --tags
```

Rebase

Switch to feature branch:

```
git switch ui
```

Rebase:

```
git rebase master
```

Switch back:

```
git switch master
```

Merge:

```
git merge ui
```

Rebase creates a cleaner commit history.

Connect to GitHub

Check remotes:

```
git remote -v
```

Add remote:

```
git remote add origin git@github.com:username/project.git
```

or

```
git remote add origin https://github.com/username/project.git
```

Push Code

First push:

```
git push -u origin main
```

Next pushes:

```
git push
```

Pull Latest Changes

```
git pull
```

Fetch without merging:

```
git fetch
```

Rename Default Branch

```
git branch -m main
```

Set default branch globally:

```
git config --global init.defaultBranch main
```

Create Markdown File

```
touch README.md
```

or

```
touch hello.md
```

Commit it:

```
git add .  
git commit -m "Add README"
```

Common Daily Workflow

```
git pull  
  
git status  
  
git add .  
  
git commit -m "Describe your changes"  
  
git push
```

Useful Git Commands

```
git status  
  
git add .  
  
git commit -m "message"  
  
git log --oneline  
  
git branch
```

```
git switch branch-name  
  
git switch -c new-branch  
  
git merge branch-name  
  
git stash  
  
git stash pop  
  
git pull  
  
git push  
  
git fetch  
  
git remote -v  
  
git tag  
  
git config --list
```

Best Practices

- Commit frequently with meaningful messages.
- Pull before starting work.
- Create feature branches instead of working directly on `main`.
- Use `.gitignore` to exclude unnecessary files.
- Keep commits focused on a single change.
- Review changes using `git diff` before committing.
- Push regularly to avoid losing work.
- Use tags for releases.
- Resolve merge conflicts carefully.
- Never commit passwords, API keys, or `.env` files.

Typical Git Development Flow

```
Create Repository
```

```
↓
```

```
git init
```

```
↓
```

Write Code



`git status`



`git add .`



`git commit`



Create Branch



Develop Feature



Merge Branch



`git push`



GitHub

Conclusion

Git is an essential tool for every software developer. By mastering the commands in this guide, you can confidently manage projects, collaborate with teams, maintain a clean version history, and deploy code safely. Practice these commands regularly to build a strong Git workflow.